
EL
ARCHIVO
PERDIDO
START

INTRODUCCIÓN

EL ARCHIVO PERDIDO ES UN JUEGO SERIO DESARROLLADO EN PYTHON UTILIZANDO LA BIBLIOTECA PYGAME, COMO PROYECTO PARA LA ASIGNATURA DE ESTRUCTURAS DE DATOS. SU PROPÓSITO ES APLICAR LOS CONCEPTOS ESTUDIADOS DURANTE EL CURSO MEDIANTE UNA EXPERIENCIA INTERACTIVA EN LA QUE EL JUGADOR DEBE RECORRER DIFERENTES SALAS DE UNA BIBLIOTECA PARA RECUPERAR LOS FRAGMENTOS DE UN CATÁLOGO QUE HA SIDO DAÑADO.

CADA SALA REPRESENTA UNA ESTRUCTURA DE DATOS Y PROPONE ACTIVIDADES RELACIONADAS CON SU FUNCIONAMIENTO. DE ESTA MANERA, EL JUGADOR INTERACTÚA CON LISTAS, TUPLAS, DICCIONARIOS Y CONJUNTOS, MIENTRAS RESUELVE DESAFÍOS QUE LE PERMITEN AVANZAR EN LA HISTORIA.

ADICIONALMENTE, EL PROYECTO INCORPORA UN ÁRBOL BINARIO DE BÚSQUEDA PARA ORGANIZAR LOS CÓDIGOS DE LOS FRAGMENTOS Y FACILITAR LA CONSULTA DEL CATÁLOGO RECONSTRUIDO. TAMBIÉN CUENTA CON UN SISTEMA DE PERSISTENCIA QUE PERMITE GUARDAR Y RECUPERAR EL PROGRESO DE LAS PARTIDAS, ASÍ COMO ALMACENAR LOS PUNTAJES OBTENIDOS

OBJETIVO GENERAL:

DESARROLLAR UN JUEGO SERIO QUE PERMITA APLICAR LOS CONCEPTOS DE ESTRUCTURAS DE DATOS MEDIANTE ACTIVIDADES INTERACTIVAS, USANDO DIFERENTES ESTRUCTURAS PARA ORGANIZAR, CONSULTAR Y ALMACENAR INFORMACIÓN.

OBJETIVOS ESPECÍFICOS:

- IMPLEMENTAR LISTAS, TUPLAS, DICCIONARIOS Y CONJUNTOS RELACIONADAS CON LA DINÁMICA DEL JUEGO.
- DESARROLLAR UN ÁRBOL BINARIO DE BÚSQUEDA PARA ORGANIZAR Y CONSULTAR LOS FRAGMENTOS DEL CATÁLOGO.
- IMPLEMENTAR MECANISMOS DE PERSISTENCIA PARA GUARDAR Y RECUPERAR EL PROGRESO DE LAS PARTIDAS.
- COMPARAR EL COMPORTAMIENTO DE LA BÚSQUEDA SECUENCIAL Y LA BÚSQUEDA MEDIANTE ÁRBOLES BINARIOS DE BÚSQUEDA.
- DISEÑAR PRUEBAS AUTOMATIZADAS QUE PERMITAN VERIFICAR EL FUNCIONAMIENTO DE LAS ESTRUCTURAS IMPLEMENTADAS Y DEL SISTEMA DE PERSISTENCIA.



ORGANIZACIÓN DEL PREOYECTO

El proyecto se desarrolló mediante una organización por archivos, en la que cada uno de estos tiene una responsabilidad específica. Esta separación permite mantener la lógica del juego independiente de las operaciones de las estructuras de datos y del almacenamiento de la información.

Los módulos principales son:

- main.py: funciona como punto de entrada para ejecutar el programa.
- juego.py: administra la ventana de Pygame, las pantallas, la navegación y el flujo de la partida.
- estructuras.py: contiene las operaciones correspondientes a listas, tuplas, diccionarios y conjuntos.
- arbol.py: implementa el árbol binario de búsqueda y las operaciones necesarias para organizar y consultar el catálogo.
- persistencia.py: gestiona el almacenamiento y la recuperación de partidas y puntajes.

Esta organización facilita la reutilización del código y permite probar los módulos de manera independiente.

IMPLEMENTACION DE TUPLAS

Las tuplas se utilizan en la sala Mapas para representar las posiciones del jugador mediante coordenadas de fila y columna.

Se eligió esta estructura ya que sus elementos no pueden modificarse directamente después de su creación. Esto permite representar una posición como un único valor que conserva sus coordenadas.

En el proyecto se implementan operaciones para crear las posiciones y separar sus valores cuando se necesitan consultar o utilizar las coordenadas.

IMPLEMENTACION DE LISTAS

LAS LISTAS SE UTILIZAN EN LA SALA ESTANTERÍAS PARA REPRESENTAR UNA COLECCIÓN DE FRAGMENTOS QUE EL JUGADOR PUEDE ADMINISTRAR.
ESTA ESTRUCTURA FUE SELECCIONADA PORQUE PERMITE ALMACENAR ELEMENTOS EN UNA SECUENCIA ORDENADA Y MODIFICAR SU CONTENIDO DURANTE LA EJECUCIÓN DEL PROGRAMA.
ENTRE LAS OPERACIONES UTILIZADAS ESTAN:

- AGREGAR FRAGMENTOS.
- BUSCAR FRAGMENTOS.
- ELIMINAR ELEMENTOS.
- ORDENAR LA COLECCIÓN.

SU UTILIZACIÓN PERMITE DEMOSTRAR CÓMO UNA LISTA FACILITA LA ADMINISTRACIÓN DE UNA COLECCIÓN DE DATOS QUE CAMBIA DURANTE EL DESARROLLO DE LA PARTIDA

IMPLEMENTACION DE DICCIONARIOS

LOS DICCIONARIOS SE UTILIZAN EN LA SALA FICHAS PARA ORGANIZAR LA INFORMACIÓN DE LOS FRAGMENTOS MEDIANTE PARES DE CLAVES Y VALORES.

CADA FRAGMENTO CONTIENE INFORMACIÓN COMO:

- CÓDIGO.
- TÍTULO.
- SALA.
- VALOR.

ESTA ESTRUCTURA PERMITE ACCEDER A LOS DATOS MEDIANTE SUS CLAVES, FACILITANDO LA CONSULTA DE INFORMACIÓN ESPECÍFICA SIN TENER QUE RECORRER TODOS LOS CAMPOS DE MANERA INDEPENDIENTE.

IMPLEMENTACION DE CONJUNTOS

LOS CONJUNTOS SE UTILIZAN PARA TRABAJAR CON COLECCIONES DE CÓDIGOS Y CONTROLAR LA PRESENCIA DE ELEMENTOS REPETIDOS.
SE ELIGIERON PORQUE ALMACENAN ELEMENTOS ÚNICOS Y PERMITEN REALIZAR OPERACIONES ENTRE DIFERENTES COLECCIONES.
EN EL PROYECTO SE UTILIZAN LAS SIGUIENTES OPERACIONES:

- ELIMINACIÓN DE DUPLICADOS: PERMITE CONSERVAR UNA SOLA APARICIÓN DE CADA CÓDIGO.
- UNIÓN: REÚNE LOS ELEMENTOS DE DOS CONJUNTOS.
- INTERSECCIÓN: OBTIENE LOS ELEMENTOS QUE TIENEN EN COMÚN.
- DIFERENCIA: IDENTIFICA LOS ELEMENTOS QUE PERTENECEN A UN CONJUNTO, PERO NO AL OTRO.

ESTAS OPERACIONES PERMITEN APLICAR LAS PROPIEDADES DE LOS CONJUNTOS A SITUACIONES RELACIONADAS CON LA RECUPERACIÓN DEL CATÁLOGO.

ALTERNATIVAS DESCARTADAS

LISTAS

ALTERNATIVA DESCARTADA: TUPLAS

SE DESCARTARON PORQUE NO PERMITEN MODIFICAR DIRECTAMENTE LA COLECCIÓN DE FRAGMENTOS.

TUPLAS

ALTERNATIVA DESCARTADA: LISTAS

SE UTILIZARON LAS TUPLAS PORQUE PERMITEN REPRESENTAR POSICIONES QUE NO NECESITAN MODIFICARSE.

DICCIONARIOS

ALTERNATIVA DESCARTADA: LISTAS

UTILIZAMOS LOS DICCIONARIOS PORQUE PERMITEN CONSULTAR INFORMACIÓN MEDIANTE CLAVES.

CONJUNTOS

ALTERNATIVA DESCARTADA: LISTAS

SE PREFIRIERON LOS CONJUNTOS PORQUE ALMACENAN ELEMENTOS ÚNICOS Y FACILITAN LAS OPERACIONES ENTRE COLECCIONES.

IMPLEMENTACIÓN DEL ÁRBOL BINARIO DE BÚSQUEDA:

EL ÁRBOL BINARIO DE BÚSQUEDA SE IMPLEMENTA EN EL MÓDULO ARBOL.PY Y TIENE COMO FINALIDAD ORGANIZAR LOS FRAGMENTOS DEL CATÁLOGO UTILIZANDO SU CÓDIGO NUMÉRICO COMO CRITERIO DE ORDENAMIENTO.

LA ESTRUCTURA ESTÁ COMPUESTA POR NODOS. CADA NODO ALMACENA UN FRAGMENTO Y REFERENCIAS A SUS POSIBLES HIJOS IZQUIERDO Y DERECHO.

PARA MANTENER LAS PROPIEDADES DEL ÁRBOL BINARIO DE BÚSQUEDA:

- LOS CÓDIGOS MENORES SE UBICAN EN EL SUBÁRBOL IZQUIERDO.
- LOS CÓDIGOS MAYORES SE UBICAN EN EL SUBÁRBOL DERECHO.
- NO SE PERMITEN CÓDIGOS DUPLICADOS.

ENTRE LAS OPERACIONES UTILIZADAS SE ENCUENTRAN:

- INSERCIÓN: INCORPORA UN FRAGMENTO EN LA POSICIÓN CORRESPONDIENTE SEGÚN SU CÓDIGO.
- BÚSQUEDA: PERMITE LOCALIZAR UN FRAGMENTO SIGUIENDO LAS REGLAS DE ORDENAMIENTO DEL ÁRBOL.
- RECORRIDO INORDEN: RECORRE LOS ELEMENTOS EN ORDEN ASCENDENTE SEGÚN SUS CÓDIGOS.
- VERIFICACIÓN DEL ORDEN: PERMITE COMPROBAR QUE LOS ELEMENTOS MANTIENEN LAS PROPIEDADES ESPERADAS.
- CÁLCULO DE ALTURA: DETERMINA LA ALTURA DEL ÁRBOL.
- RECONSTRUCCIÓN DEL CATÁLOGO: PERMITE ORGANIZAR LOS FRAGMENTOS RECUPERADOS MEDIANTE EL ÁRBOL.

LA IMPLEMENTACIÓN PERMITE ESTUDIAR CÓMO LA ORGANIZACIÓN DE LOS DATOS INFLUYE EN EL COMPORTAMIENTO DE LAS BÚSQUEDAS.



TABLA COMPARATIVA

	Busqueda secuencial	Arbol equilibrado	altura equilibrada	arbol degenerado	altura degenerada
10 fragmentos	5.50	2.90	4	5.50	10
100 fragmentos	50.50	50.80	7	50.50	100
1000 fragmentos	500.50	8.99	10	500.50	1000

¡NTERPRETACiON DE RESULTADOS

- Árbol equilibrado : Al insertar primero el dato del medio de cada grupo, el árbol queda bien repartido entre izquierda y derecha. Por eso, en 1000 fragmentos solo tiene 10 niveles de altura y encuentra cualquier libro en unos 9 pasos en promedio, mientras que buscar en la lista toma 500 pasos.

- Árbol degenerado (lento): Si insertamos los códigos ordenados de menor a mayor (1, 2, 3...), cada número nuevo es mayor que el anterior y se va siempre hacia la derecha. El árbol deja de tener ramas y se vuelve una fila recta de 1000 niveles, tardando los mismos 500.50 pasos que una lista normal.

El módulo persistencia.py guarda la partida en data/partida.json y los récords en data/puntajes.json para que no se pierda el progreso al cerrar el juego.
Además, revisamos todo el código con pruebas automáticas (unittest):

- test_estructuras.py: comprueba que las listas, tuplas, diccionarios y conjuntos funcionen bien y rechacen datos inválidos.
- test_arbol.py: revisa que el árbol ordene los códigos, no acepte repetidos y cuente bien las comparaciones.
- test_persistencia.py: usa una carpeta temporal para probar que se pueda guardar y cargar la partida, agregar puntajes y manejar archivos que no existen o están dañados sin borrar los datos reales del juego.

LIMITACIONES Y MEJORAS

Limitaciones:

- Cantidad limitada de salas y actividades.
- Posibilidad de que el árbol binario de búsqueda se vuelva degenerado.
- Opciones de interacción y elementos visuales que podrían ampliarse.
- Almacenamiento de datos mediante archivos locales.

Posibles mejoras:

- Incorporar nuevos niveles y desafíos.
- Implementar árboles autoequilibrados.
- Agregar animaciones, sonidos y nuevas interacciones.
- Mejorar el almacenamiento y la administración de partidas.

USO DE IA

SE UTILIZARON HERRAMIENTAS DE INTELIGENCIA ARTIFICIAL DURANTE EL PROYECTO, ESPECÍFICAMENTE PARA LA REVISIÓN Y DETECCIÓN DE ERRORES EN EL CÓDIGO. ADICIONALMENTE, SE EMPLEÓ CURSOR AGENT PARA ASISTIR EN LA CREACIÓN DE COMMITS EN MOMENTOS EN QUE EL PROCESO MANUAL DESDE LA TERMINAL NOS GENERABA INCONVENIENTES Y ERRORES TÉCNICOS, LOS CUALES FUERON RESUELTOS POSTERIORMENTE POR EL EQUIPO.